

AWS Glue, Crawler & Redshift

A clear, practical guide with real-world examples & architecture

S3 · Glue ETL · Crawler/Data Catalog · Redshift (MPP, DISTKEY/SORTKEY, Spectrum) | the ShopFast example

A read-it-and-understand guide with real-world examples and architecture diagrams. No prior AWS depth assumed. We'll use one running example — a **retail company ("ShopFast") building an analytics platform** — so every service has a concrete job.

The big picture first (why these services exist)

Imagine ShopFast has data everywhere: orders in a database, clickstream logs in files, product info in spreadsheets. Leadership wants dashboards: "revenue by region", "top products", "repeat customers". To answer those questions you need to **collect** the data, **clean and reshape** it, **describe** it so tools can read it, and **store** it somewhere you can run fast SQL. That is exactly the job of these services:

- **Amazon S3** — cheap, unlimited storage. The "data lake" where raw and processed files live.
- **AWS Glue** — the serverless **ETL engine** that cleans/transforms data (runs Apache Spark).
- **AWS Glue Crawler + Data Catalog** — automatically **discovers the schema** of your files and records it in a central catalog, so query tools know what columns/types exist.
- **Amazon Redshift** — a **data warehouse**: a database built for fast analytics over billions of rows, powering BI dashboards.

Think of it as a kitchen: **S3** is the pantry (raw ingredients), **Glue** is the chef (prepares dishes), the **Crawler/Catalog** is the labeled menu (what's available), and **Redshift** is the dining room where meals are served fast to many guests.

PART 1 — AWS GLUE

1.1 What Glue actually is

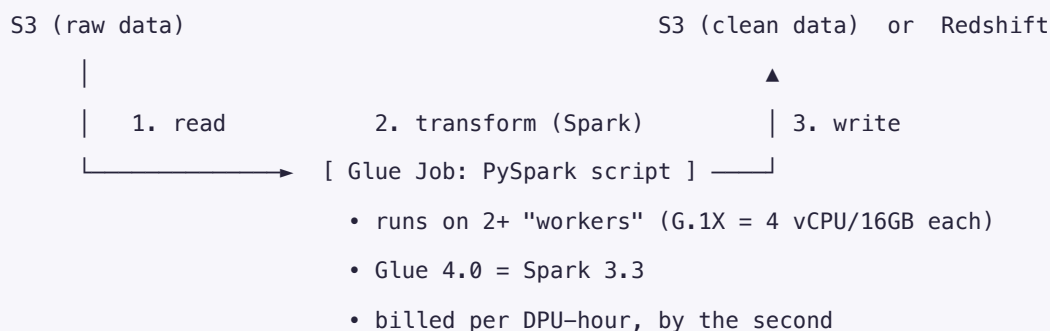
AWS Glue is a **serverless data-integration service**. "Serverless" means you don't create or manage servers — you hand Glue your transformation code and it spins up a temporary Apache **Spark** cluster, runs the job, and tears it down. You pay only for the seconds it ran.

Glue is really **three things bundled** under one name (people constantly mix these up):

1. **Glue ETL Job** — the compute that *transforms* data (Spark or Python).
2. **Glue Data Catalog** — a *metadata* store: databases → tables → column definitions.
3. **Glue Crawler** — a process that *scans* your data and fills the catalog automatically.

Plus supporting pieces: **Glue Studio** (a visual drag-and-drop job builder), **Triggers & Workflows** (orchestration), **Connections** (to databases), and **Job Bookmarks** (process only new data).

1.2 How a Glue ETL job works (the flow)



Extract → Transform → Load (ETL):

- **Extract:** read source files (S3, a database via a Connection, etc.).
- **Transform:** clean/reshape with Spark — filter bad rows, join datasets, flatten nested JSON, deduplicate, aggregate, convert CSV/JSON → **Parquet** (columnar, compressed).
- **Load:** write the result to S3 (as Parquet) or straight into Redshift.

Real example — ShopFast orders (PySpark)

Raw orders land in `s3://shopfast-lake/raw/orders/` as messy JSON. This Glue job cleans them and writes Parquet to the curated zone:

```

import sys

from awsglue.context import GlueContext
from pyspark.context import SparkContext
from pyspark.sql.functions import col, to_date

glueContext = GlueContext(SparkContext())
spark = glueContext.spark_session

# 1. EXTRACT
orders = spark.read.json("s3://shopfast-lake/raw/orders/")

# 2. TRANSFORM: keep valid orders, clean types, add an order_date column
clean = (orders
    .filter(col("status") != "cancelled")          # drop cancelled
    .filter(col("amount") > 0)                     # drop bad amounts
    .withColumn("amount", col("amount").cast("double"))
    .withColumn("order_date", to_date(col("created_at")))
    .dropDuplicates(["order_id"]))                # dedupe

# 3. LOAD: write partitioned Parquet (partitioning speeds up later queries)
(clean.write
    .mode("overwrite")
    .partitionBy("order_date")                     # creates .../order_date=2026-08-07/
    .parquet("s3://shopfast-lake/curated/orders/"))

```

Why Parquet + partitioning? Parquet stores data by *column* and compresses it, so analytics tools scan far less data. Partitioning by `order_date` lets a query for "last week" skip every other day's files entirely — faster and cheaper.

1.3 Key Glue concepts you should know

- **DPU (Data Processing Unit):** the unit of compute (≈ 4 vCPU + 16 GB). You pick a **worker type** (`G.1X`, `G.2X`) and a **number of workers**. More workers = faster + more cost. Billing is per DPU-hour, per second (1-min minimum).
- **Job Bookmarks:** Glue can remember what it already processed and handle **only new files** on the next run — essential for daily incremental pipelines (so you don't reprocess years of data every night).
- **DynamicFrame vs DataFrame:** Glue adds a `DynamicFrame` (schema-flexible, good for messy/semi-structured data and bookmarks); you can convert to a normal Spark `DataFrame` for standard SQL-style transforms.

- **Triggers & Workflows:** schedule jobs (cron), chain them (job A → crawler → job B), or fire on an event (new file arrives via EventBridge).
- **Glue Studio:** build jobs visually (Source → Transform → Target nodes) without writing Spark by hand — great for learning and for simple pipelines.

1.4 When to use Glue (and when not)

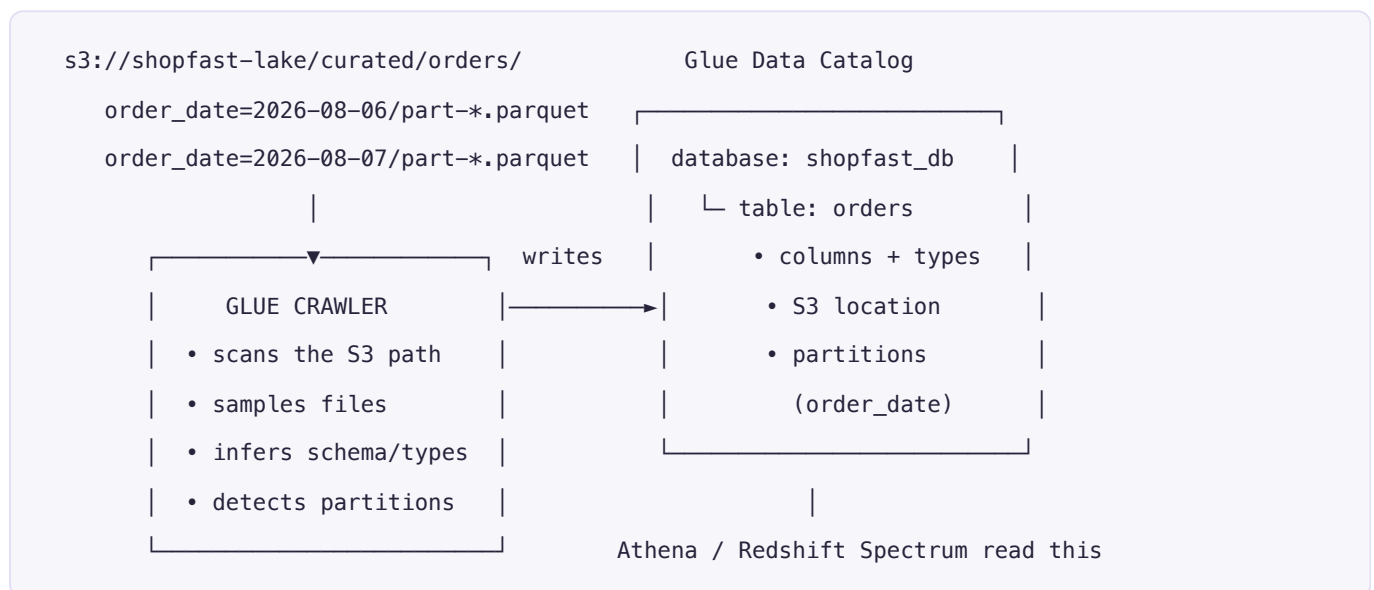
Use Glue when you want **serverless, pay-per-use ETL** without managing clusters — perfect for batch transformation and cataloging in a data lake. Reach for **Amazon EMR** instead when you need long-running, heavily-customized Spark with special libraries or fine-grained cluster control. For simple visual data-prep by analysts, there's **Glue DataBrew**.

PART 2 — AWS GLUE CRAWLER & DATA CATALOG

2.1 The problem the crawler solves

You have thousands of Parquet files in `s3://shopfast-lake/curated/orders/`. Athena or Redshift Spectrum want to run SQL like `SELECT FROM orders`, *but they need to know*: what columns exist, what types, where are the files, how is it partitioned?* Manually writing that schema is tedious and breaks when data changes. The **crawler automates it**.

2.2 What a crawler does, step by step



1. **Scans** the S3 location (a "data store" target).
2. Uses **classifiers** to recognize the format (Parquet, JSON, CSV, Avro, ORC...) — built-in classifiers cover common formats; you can add **custom classifiers** (e.g., a Grok pattern for odd log files).
3. **Infers the schema** — column names and data types — by sampling files (Parquet/ORC are self-describing, so this is exact; CSV is guessed).
4. **Detects partitions** from the folder structure (`order_date=...` becomes a partition column).
5. **Writes/updates a table** in the **Glue Data Catalog** (a database → table entry). No data is moved — only *metadata* is recorded.

2.3 The Data Catalog — one schema, many engines

The catalog is a **Hive-compatible metastore**. Because it's a shared, central place, the *same* table definition is readable by **Athena**, **Redshift Spectrum**, **EMR**, and Glue jobs. Define once, query from anywhere. That's the whole value: a single source of truth for "what data do we have and what does it look like."

2.4 Real example — cataloging ShopFast orders (CLI)

```
aws glue create-database --database-input '{"Name":"shopfast_db"}'

aws glue create-crawler --name orders-crawler \
  --role AWSGlueServiceRole-shopfast \
  --database-name shopfast_db \
  --targets '{"S3Targets":[{"Path":"s3://shopfast-lake/curated/orders/"}]}'

aws glue start-crawler --name orders-crawler
# after it finishes:
aws glue get-tables --database-name shopfast_db --query 'TableList[].Name'
# -> ["orders"] with columns, types, and an order_date partition
```

Now `SELECT * FROM shopfast_db.orders WHERE order_date = DATE '2026-08-07'` works in Athena.

2.5 Practical crawler wisdom (interview + real-world)

- **Schedule vs on-demand:** run on-demand while developing; schedule (e.g., hourly/daily) in production to pick up new partitions automatically.
 - **When to skip crawlers:** for stable schemas, many teams **define tables explicitly** (DDL) and use **partition projection** — no crawler runs, exact control, no mis-inferred types, and it's version-controllable.
 - **Common gotchas:** crawlers can mis-guess CSV types (everything as string), create odd tables if a folder mixes formats, and add cost/latency if scheduled too often. Point a crawler at a *clean, single-format* prefix.
-

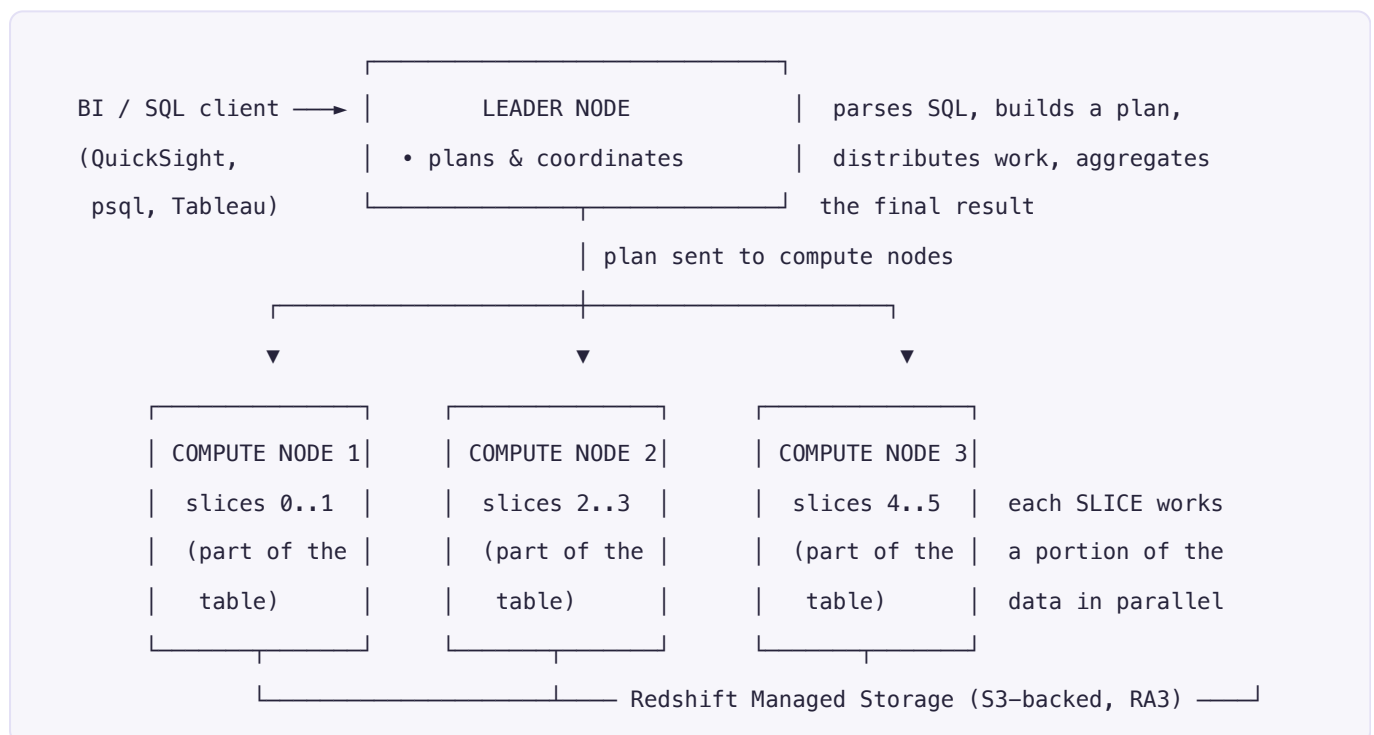
PART 3 — AMAZON REDSHIFT

3.1 What Redshift is

Amazon Redshift is a **fully-managed, petabyte-scale data warehouse**. A data warehouse is a database optimized not for lots of small transactions (that's OLTP, like MySQL) but for **analytics** (OLAP) — scanning and aggregating huge tables to answer business questions. Redshift makes this fast with two big ideas: **columnar storage** and **massively parallel processing (MPP)**.

- **Columnar storage:** it stores each *column* together on disk. A query like `SELECT SUM(amount) FROM orders` only reads the `amount` column, skipping the rest — huge I/O savings vs a row store.
- **MPP:** it splits your data across many machines ("compute nodes") and runs the query on all of them in parallel, then combines results.

3.2 Redshift architecture (draw this)



- **Leader node:** the "brain" — receives SQL, creates an optimized query plan, and coordinates the compute nodes. You don't store data here.
- **Compute nodes:** do the actual work; each is divided into **slices** (one per CPU core). Data is spread across slices so work happens in parallel.
- **Node types:**
 - **RA3** (modern default) — compute and storage are **separated**; data lives in **Redshift Managed Storage** (backed by S3), so you scale compute without moving data.
 - **DC2** — older, storage is local to the node (good for small, performance-sensitive sets).

- **Redshift Serverless:** no cluster to size at all — it auto-provisions capacity and you pay per usage (measured in **RPUs**). Great for spiky or unpredictable workloads and for getting started.

3.3 The three tuning ideas that make Redshift fast

These come up in every Redshift interview and matter in real life:

1. Distribution style (how rows are spread across slices):

- `DISTKEY(col)` — rows with the same key value go to the same slice. Use on the column you **join on** most, so joined rows sit together (no network shuffle).
- `DISTSTYLE ALL` — copy the whole table to every node. Use for **small dimension tables** (e.g., a `regions` lookup) so joins are local.
- `DISTSTYLE EVEN` — spread rows round-robin. A safe default when there's no obvious join key.

2. Sort key (how rows are ordered on disk):

- `SORTKEY(col)` — usually a **date/timestamp**. Queries filtering by that column skip huge blocks of data ("zone maps"), so `WHERE order_date BETWEEN ...` is very fast.

3. Compression (encoding): Redshift compresses each column automatically (`ENCODE AUTO`), shrinking storage and I/O.

Real example — ShopFast fact + dimension tables

```
-- Big fact table: distribute by the join key, sort by date
CREATE TABLE orders (
  order_id      BIGINT,
  customer_id   BIGINT,
  product_id    BIGINT,
  amount        DECIMAL(10,2),
  order_date    DATE
)
DISTKEY(customer_id)      -- joins to customers stay on the same slice
SORTKEY(order_date);      -- date-range filters are fast

-- Small lookup table: copy to every node so joins are local
CREATE TABLE regions (
  region_id     INT,
  region_name   VARCHAR(50)
)
DISTSTYLE ALL;
```

3.4 Getting data IN and OUT

Load with COPY (the fast, parallel way — never insert row-by-row):

```
COPY orders
FROM 's3://shopfast-lake/curated/orders/'
IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftLoadRole'
FORMAT AS PARQUET;
```

COPY reads many files in parallel across all slices — this is how you load millions of rows quickly.

Where does that S3 data come from? Your Glue job produced it. This is the join between the two worlds.

Export with UNLOAD (write query results back to S3, e.g., for sharing or archiving):

```
UNLOAD ('SELECT * FROM orders WHERE order_date = ''2026-08-07''')
TO 's3://shopfast-lake/exports/orders_'
IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftLoadRole'
FORMAT AS PARQUET;
```

3.5 Redshift Spectrum — query S3 directly, no loading

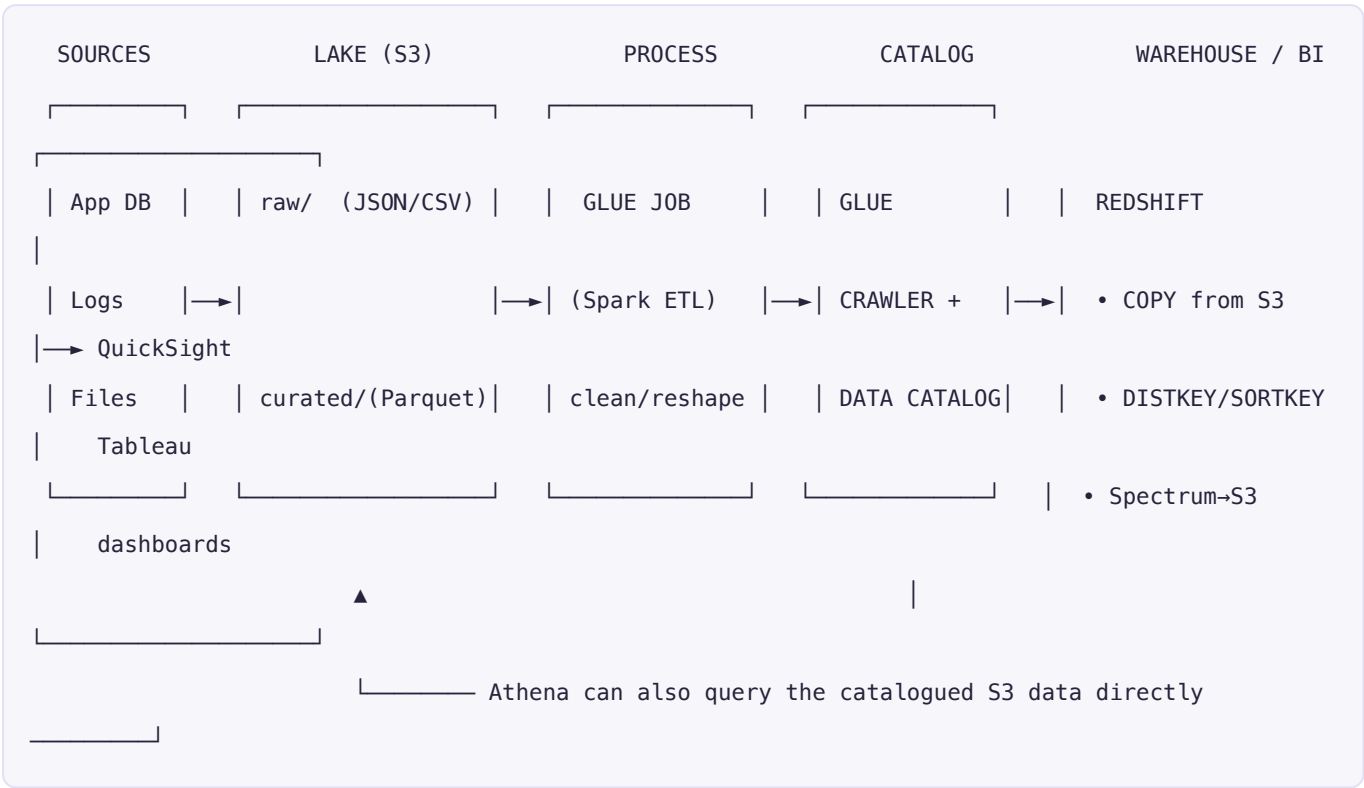
Sometimes you don't want to load everything into Redshift. **Spectrum** lets Redshift run SQL **directly on S3 files**, using the **Glue Data Catalog** for schema (the same catalog your crawler filled!). You create an **external schema** pointing at the catalog, then query external tables and even **join them to tables inside Redshift**:

```
CREATE EXTERNAL SCHEMA spectrum_lake
FROM DATA CATALOG DATABASE 'shopfast_db'
IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftSpectrumRole';

-- join cold data in S3 (via Spectrum) to hot data in Redshift
SELECT r.region_name, SUM(o.amount) AS revenue
FROM   spectrum_lake.orders o          -- lives in S3, catalogued by the crawler
JOIN   regions r ON r.region_id = o.region_id -- lives in Redshift
GROUP BY r.region_name;
```

This is the "lakehouse" pattern: keep huge/cold data cheaply in S3, keep hot/frequently-used data in Redshift, and query across both.

PART 4 — HOW IT ALL FITS TOGETHER (end-to-end architecture)



The story, one sentence: raw data lands in **S3**, **Glue** cleans it into Parquet, a **Glue Crawler** catalogs it in the **Data Catalog**, and **Redshift** either loads it (**COPY**) for fast BI or queries it in place with **Spectrum** — with **Athena** available for ad-hoc SQL on the same catalog.

PART 5 — CHOOSING BETWEEN TOOLS (quick decision help)

Athena vs Redshift — both run SQL on your data, but:

- **Athena** = serverless, pay-per-query on S3 files. Best for *ad-hoc* queries, occasional analysis, and exploring the lake. No infrastructure.
- **Redshift** = a provisioned/serverless warehouse. Best for *frequent, complex, high- concurrency* BI dashboards and large joins where consistent fast performance matters.
- Rule of thumb: **occasional S3 queries** → **Athena**; **a busy BI platform** → **Redshift** (and use **Spectrum** to bridge Redshift to S3).

Glue vs EMR — Glue = serverless, managed Spark for standard ETL + cataloging. EMR = full control over big, custom, long-running Spark/Hadoop clusters.

Crawler vs explicit DDL — crawler for unknown/changing schemas; explicit `CREATE TABLE` + partition projection for stable production schemas.

PART 6 — KEY TAKEAWAYS (say these confidently)

- **S3 = storage, Glue = compute/ETL, Catalog = schema, Redshift = warehouse.** Storage and compute are decoupled — scale and pay for each independently.
- **Glue is serverless Spark** billed per DPU-second; use **bookmarks** for incremental loads.
- **The crawler only writes metadata**, not data; the **Data Catalog** is a shared schema used by Athena, Redshift Spectrum, and EMR.
- **Redshift is columnar + MPP**; performance comes from **DISTKEY** (co-locate joins), **SORTKEY** (skip blocks on filters), and **compression**.
- **COPY** loads from S3 in parallel; **UNLOAD** writes back; **Spectrum** queries S3 in place via the Glue Catalog — the lakehouse pattern.
- **RA3** separates compute/storage; **Redshift Serverless** removes cluster sizing entirely.